

Marcador Electrónico para Natación — Firmware CircuitPython

Firmware principal del marcador electrónico de natación: un panel LED matricial que muestra tiempos, cronómetro y puntuaciones en tiempo real vía MQTT. Está basado en **CircuitPython** sobre una placa **Adafruit MatrixPortal S3 (ESP32-S3)** con pantalla LED RGB de **128x64**.

Este repo es la versión completa del firmware (con módulos de cronómetro, RTC, gestión de tópicos y persistencia de datos). El repo `marcador-electronico-para-natacion` contiene la versión prototipo.

Configuración (`settings.toml`)

Antes de usar, edita el archivo `settings.toml` en la raíz del dispositivo (misma carpeta donde vive `code.py`):

```
CIRCUITPY_WIFI_SSID = "red_wifi"
CIRCUITPY_WIFI_PASSWORD = "contraseña_wifi"

MQTT_BROKER = "bad771df41604666a6a56e78d7184c9c.s1.eu.hivemq.cloud"
MQTT_PORT = 8883
MQTT_USERNAME = "usuario_mqtt"
MQTT_PASSWORD = "contraseña_mqtt"
MQTT_SSL = "true"
```

Variable	Descripción
<code>CIRCUITPY_WIFI_SSID</code>	Nombre de la red WiFi
<code>CIRCUITPY_WIFI_PASSWORD</code>	Contraseña de la red WiFi
<code>MQTT_BROKER</code>	Dirección del broker MQTT (ej. HiveMQ Cloud)
<code>MQTT_PORT</code>	Puerto MQTT (<code>1883</code> sin SSL, <code>8883</code> con SSL)
<code>MQTT_USERNAME</code>	Usuario del broker MQTT
<code>MQTT_PASSWORD</code>	Contraseña del broker MQTT

Variable	Descripción
<code>MQTT_SSL</code>	<code>"true"</code> para usar SSL/TLS, <code>"false"</code> para conexión sin cifrar

Nota de seguridad: no subas credenciales reales a repositorios públicos. Usa variables de entorno o borra los secretos antes de hacer commit.

Hardware Requerido

- Adafruit MatrixPortal S3 con ESP32-S3 y CircuitPython 10
- Matriz LED RGB 128x64 (configurada como 2 tiles de 64x32)
- Fuente de alimentación de 5V y amperaje suficiente para la matriz
- Módulo RTC DS1307
- Sensor LM35 para captar temperatura ambiente
- Botones pulsadores

Dependencias (librerías CircuitPython)

Todas están incluidas en la carpeta `lib/`:

- `adafruit_matrixportal` — control de la matriz LED
- `adafruit_minimqtt` — cliente MQTT
- `adafruit_display_text` — renderizado de texto en pantalla
- `adafruit_display_shapes` — formas geométricas
- `adafruit_imageload` — carga de imágenes
- `adafruit_esp32spi` — comunicación con el coprocesador WiFi
- `adafruit_connection_manager` / `adafruit_requests` — gestión de red
- `adafruit_ds1307` — RTC DS1307 (opcional)
- `adafruit_io` — cliente Adafruit IO (opcional)
- `asyncio` — soporte asíncrono básico

Funcionamiento

Pantalla principal (modo `show`)

Muestra 3 líneas:

```
12:30 PM      HEAT 28
RECORDS 1.01:23:45.67
          2.00:12:34.56
```

- **Línea 1:** Hora (formato 12h), indicador AM/PM y temperatura
- **Línea 2-3:** Hasta 2 récords visibles; si hay más de 2, cambia cada 5 segundos

Pantalla de cronómetro (modo `chrono`)

Al enviar `esp32s3/screen_type` con valor `chrono` :

```
12:30 PM - 28°
Round|HH|MM|SS|MS
1   |01|23|45|67
```

Cronómetro

Controlado mediante el tópico `esp32s3/chrono` :

- `play_chrono:<id>` — inicia el cronómetro con el número de ronda indicado
- `pause_chrono:<id>` — detiene el cronómetro y guarda el tiempo como puntuación

Ciclo principal (`code.py`)

1. Conecta WiFi
2. Conecta al broker MQTT
3. Publica el estado como `online` en `esp32s3/status`
4. Escucha mensajes MQTT y actualiza hora, color, temperatura, récords y cronómetro
5. Publica la hora actual cada minuto en `esp32s3/send_time`
6. Publica el estado del cronómetro cada segundo mientras esté activo en `esp32s3/send_chrono`
7. Actualiza la pantalla LED continuamente

Tópicos MQTT

Suscripciones (el display escucha)

Tópico	Formato	Descripción
esp32s3/settime	HH:MM	Ajusta la hora del display
esp32s3/settemp	28	Establece la temperatura mostrada
esp32s3/setcolor	#RRGGBB	Cambia el color del texto
esp32s3/new_user	{"id":"1","nombre":"..."}	Agrega un nuevo nadador
esp32s3/get_users	(cualquier texto)	Solicita todos los usuarios y puntuaciones
esp32s3/chrono	play_chrono:<id> o pause_chrono:<id>	Control del cronómetro
esp32s3/get_scores	(cualquier texto)	Solicita las puntuaciones guardadas
esp32s3/del_score	<id>	Elimina la puntuación de un nadador
esp32s3/del_record	<id>	Elimina un nadador y su puntuación
esp32s3/screen_type	show o chrono	Cambia el modo de pantalla

Publicaciones (el display envía)

Tópico	Descripción
esp32s3/status	online / offline
esp32s3/send_time	Hora actual y temperatura (cada minuto)
esp32s3/send_chrono	Estado del cronómetro (cada segundo si activo)
esp32s3/user_data	Lista completa de usuarios y puntuaciones
esp32s3/send_scores	Puntuaciones al detener el cronómetro
esp32s3/send_color	Color actual en hex
esp32s3/send_view	Datos visibles actualmente en pantalla

Archivos del proyecto

```
├─ boot.py                # Habilita escritura en el sistema de archivos
├─ code.py                # Punto de entrada (llama a utils.main.main())
├─ settings.toml          # Configuración WiFi y MQTT
├─ data.txt               # Datos persistentes (color, scores, temp, etc.)
├─ fonts/                 # Fuentes .bdf para la matriz LED
│   ├── 12-Fixed-SemiCond.bdf
│   ├── 12-Sazanami-Mincho.bdf
│   └── 12-Terminus.bdf
├─ lib/                   # Librerías CircuitPython
│   ├── adafruit_matrixportal/
│   ├── adafruit_minimqtt/
│   ├── ...
│   └── utils/             # Código fuente de la aplicación
│       ├── __init__.py
│       ├── main.py         # Bucle principal
│       ├── config.py       # Configuración desde settings.toml
│       ├── mqtt_client.py  # Cliente MQTT con callbacks
│       ├── screen.py       # Control de la matriz LED
│       ├── display_content.py # Generación del contenido a mostrar
│       ├── topic_manager.py # Manejo de tópicos MQTT entrantes
│       ├── chrono.py       # Cronómetro
│       ├── rtc.py          # Reloj en software
│       ├── time_updates.py  # Publicación periódica de la hora
│       └── file_system.py   # Lectura/escritura en data.txt
└─ sd/                   # Directorio para tarjeta SD (sin usar)
```

Personalización

Tamaño de pantalla

En `lib/utils/screen.py` se define la matriz:

```
matrix = Matrix(width=128, height=64, bit_depth=1, tile_rows=2)
```

Se ajustan `width`, `height` y `tile_rows` según tu configuración de paneles LED.

Espaciado de líneas

En `lib/utils/display_content.py` se usa `line_spacing=10` para la pantalla principal. Este valor se cambia si usas una matriz de distinto tamaño.

Relación con los otros repositorios

- **CircuitPython** (este repo): firmware completo del marcador.
- **Frontend** (`marcador-electronico-para-natacion-frontend`): interfaz web de control vía MQTT.
- **General** (`marcador-electronico-para-natacion`): prototipo inicial del firmware.